

Contributing to the dashboard

How to get your work in front of the team and then in front of stakeholders — from cloning the repo to seeing your change land in Prototypes — without needing anyone to hand-hold a deploy.

Last updated 18 September 2026 · Owner: Jillienne Moreno · Applies to `jillmoreno/xcel-ux-dashboard`

CONTENTS

1. The big picture
2. What you need before you start
3. The nine steps
4. Two kinds of work, two homes
5. Feature flags — the one rule
6. The Refinement form, field by field
7. Things not to do
8. When something looks wrong
9. Glossary

01 The big picture

The dashboard is **one codebase deployed to two websites**. Every `git push` to `main` rebuilds both.

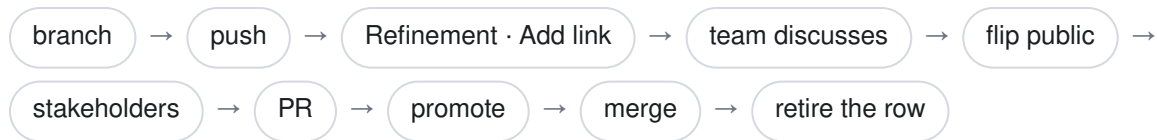
SITE	WHO OPENS IT	WHAT IT SHOWS
The full site	You, Jillienne, the team. Behind a Netlify password.	Everything — Prototypes, Refinement, Other Links, Research, and the whole <i>UX & Dev Access</i> group (Design, Exploration, Sandbox, Development, Done, Archive, QA Notes, To Do, and this guide).
The public site	Stakeholders, with the link. No password.	Only Prototypes, Refinement, Other Links and Research. The rest is not locked — it is <i>absent</i> . Refinement shows only the rows someone has flipped to public, and is read-only there.

Two sections do the work of getting a design in front of people, and they are different acts:

- **Refinement** is the review inbox. It is a list of links you add *on the page* — no code, no commit. A row points at your branch's own build (or an HTML file on it). The team sees every row; stakeholders see a row only after its *Show on public site* toggle is on.

- **Prototypes** is the product as it stands — the live React build at the flag defaults committed on `main`. Only a merge to `main` changes it, and only Jillienne decides what merges into that baseline.

So the loop is:



The rest of this guide walks each of those, in order, with the exact commands and the exact fields.

02 What you need before you start

- **Access to the GitHub repo** `jillmoreno/xcel-ux-dashboard` with permission to push branches. Ask Jillienne if you cannot see it.
- **Node.js 20 or newer** and **git** installed. Check with `node -v` and `git --version` in Terminal.
- **Claude Code**, for working in the repo. The repo carries its own instructions (`CLAUDE.md`) and skills (`.claude/skills/`), so a fresh Claude Code session already knows the project's rules.
- **The full site's address and password**. Ask Jillienne — it is not written down here on purpose.

YOU DO NOT NEED

A Netlify account, any deploy tooling, or to know how the two sites are configured. Pushing a branch is the deploy. Everything else is set up once, centrally, and described in the README for whoever maintains it.

03 The nine steps

1 Clone the repo and run it

You · once

Clone means "download the project as a git repository". You do this once; after that you pull updates rather than re-downloading.

```
git clone https://github.com/jillmoreno/xcel-ux-dashboard.git
cd xcel-ux-dashboard
npm install
npm run dev
```

Open <http://localhost:5200>. You should see the dashboard. The port is **5200** on purpose — the sibling dashboards use 5180 and 5190, and they look alike, so a wrong port shows you the wrong project with no error.

Refinement and Other Links will say their endpoint is unreachable when you run `npm run dev`. That is expected — those two lists live on Netlify, and plain `vite` does not serve them. You add rows on the full site, not locally.

2 Make a branch

You · every piece of work

A branch is your own copy of the code to work on. Nothing you do on it touches what anyone else sees until it is merged.

```
git checkout main
git pull
git checkout -b feat/study-streak-card
```

Naming: lowercase, words joined with dashes, a short prefix that says what kind of thing it is — `feat/`, `explore/`, `fix/`. The branch name becomes part of a web address in step 5, and slashes turn into dashes there, so `feat/study-streak-card` becomes `feat-study-streak-card`.

3 Do the work in Claude Code

You · with Claude

Open the repo folder in Claude Code and describe what you are building. Three things make this go smoothly:

- **Tell Claude which kind of work it is** — a React surface of the product, or a standalone HTML exploration. Section 04 explains the difference; it decides where the files go.
- **Ask Claude to put new product work behind a feature flag**, and to set that flag's default *on* for your branch. Section 05 explains why. Claude knows the flag catalog (`src/context/FeatureFlagContext.tsx`).
- **Let Claude read `CLAUDE.md` first**. It is the project's memory — every decision, every trap, every "we tried that and here is why not". A change that contradicts it will be questioned in review.

Commit as you go. Small commits with plain-English messages are easier for everyone to follow than one big one at the end.

```
git add -A
git commit -m "feat: study streak card on Home, behind dashboard-study-streak"
```

4 Check it before you push

You · before every push

Run the three checks. They are the same ones review will run, so a green result here means no surprises later.

```
npx tsc -b --noEmit      # types
npx vitest run           # the test suite (~800 tests, about a minute)
npm run lint             # code style
```

If you edited anything under `public/prototypes/`, also run `npm run smoke` — those pages have their own suites and `vitest` does not run them.

A failing test is usually the suite telling you a decision is recorded somewhere — for example, the nav order, or which rows sit on the public front door. Read the test's comment; it will say what was decided and why. If your change is meant to re-decide it, update the test in the same commit and say so in the message.

5 Push — that is the deploy

You · Netlify does the rest

```
git push -u origin feat/study-streak-card
```

Netlify notices the new branch and builds it as its own website, with no password, at:

```
https://feat-study-streak-card--ux-demo-xceldashboard.netlify.app
```

That is `<your-branch-with-dashes>--ux-demo-xceldashboard.netlify.app`. The build takes two to three minutes. Every later push to the same branch updates the same address.

The link to share is the product, in demo mode:

```
https://feat-study-streak-card--ux-demo-xceldashboard.netlify.app/dashboard-rebrand?demo=
```

`?demo=1` makes the page show your branch's *committed* flag defaults — what you set in step 3 — rather than whatever toggles the person opening it last left in their own browser. Without it, a reviewer may see their own leftover settings and think that is your design.

KNOW THIS

A branch address is **not private**. Anyone who has it can open it, and every pushed branch gets one. That is what makes stakeholder review possible without a password — and it is why nothing confidential goes on a branch.

6 Add it to Refinement

You · on the full site

Open the **full** site, click **Refinement** in the left nav, then **Add link** (or **Add the first link** if the list is empty). Paste the address from step 5, give it a title, and write a one-line note saying where to look. Leave **Show on public site** off. Section 06 has every field.

The row appears immediately for everyone on the full site. It is saved to a shared store, not to your browser, so the team sees it the moment you save.

7 Discuss, iterate, push again

You and the team

The team opens your row, looks, and talks — wherever the team talks. You make changes on the branch, run the checks, push. The address does not change, so the Refinement row is still right.

If what the row *is* changes, use **Edit** on the row to update its title or note.

There is deliberately no comment thread on the row. Refinement is a list of things to look at, not a place to record the discussion; that stays where the team already talks.

8 Flip it public when it is ready for stakeholders

Jillienne — or you, once agreed

Edit the row and switch **Show on public site** on. From that moment the row shows on the public site's Refinement list, read-only, and stakeholders with the public link can open your branch build. Switch it off again to take it back out — nothing is deleted.

The convention is that Jillienne flips it, because the public list is what stakeholders read as "what the team wants us to look at". If she has asked you to flip your own, go ahead.

9 Open a pull request → promote → merge → retire the row

You open the PR · Jillienne promotes and merges

When the work is agreed, open a pull request from your branch to `main` on GitHub. Jillienne then runs the `promote-to-prototype` skill *on your branch, before merging*. It lists every feature flag your branch changed and asks her, flag by flag, whether that default ships in Prototypes. Approved flags keep your "on"; declined ones are set back to `main`'s value, so the work lands in the sandbox sections of the full site without reaching the public product.

She merges. Both sites rebuild. The last step is retiring your Refinement row — **Remove** on the row — because the thing it pointed at is now in Prototypes (or the branch is gone).

DONE LOOKS LIKE

Your flag's default is on in `main`, the Prototypes section shows your work at `/dashboard-rebrand?demo=1`, and Refinement no longer lists your branch.

04 Two kinds of work, two homes

Where files go depends on what you are making, and picking wrong is the most common mistake in this repo.

YOU ARE MAKING	IT LIVES IN	REVIEWERS OPEN	PROMOTED BY
A React surface of the product	<code>src/</code> — a page in the shell, a card, a panel, a variant of an existing block. Behind a feature flag.	<code><branch>-...</code> <code>netlify.app/</code> <code>dashboard-</code> <code>rebrand?</code> <code>demo=1</code>	Flipping the flag's default on <code>main</code> (the promote skill).
A standalone HTML exploration	<code>public/</code> <code>demos/<your-</code> <code>file>.html</code> — a hand-authored page answering a brief, not wired into the app.	<code><branch>-...</code> <code>netlify.app/</code> <code>demos/<your-</code> <code>file>.html</code>	Moving it to <code>public/</code> <code>prototypes/</code> with a row in <code>prototypeFeatures.ts</code> if it is a finished exploration, or rebuilding it in <code>src/</code> if it became a product surface.

WHY `PUBLIC/DEMOS/` AND NOT `PUBLIC/PROTOTYPES/`

The public site 404s everything under `/prototypes/` at the edge — that folder holds the finished FinServ explorations, which are gated. A file there is invisible on your branch's public build too. `public/demos/` is deliberately left open for exactly this: HTML in review. Nothing should live there long, and nothing there gets a row in the feature list — the Refinement panel is its listing.

05 Feature flags — the one rule

Every change to the product starts **invisible to Prototypes**: behind a flag whose default on `main` is off (or set to the existing variant). That is what lets many designers work on the same product without anyone's half-finished idea appearing in front of stakeholders.

On **your branch**, you set that flag's default *on*.

That is what makes your branch build show your work at `?demo=1`. It also means the promote skill can

see, mechanically, exactly which defaults your branch changed — the list it asks Jillienne about in step 9 is the diff of the flag catalog between your branch and `main`.

- Flags live in `src/context/FeatureFlagContext.tsx`, in `FEATURE_FLAGS`. Ask Claude to add yours there and to add it to the `/dashboard-rebrand` panel scope so reviewers can toggle it.
- Reviewers can flip flags live on your build: hover the prototype bar and find the robot icon, which opens the flag sheet. In demo mode those flips are temporary and reset on exit.
- Do not promote by adding a row to Prototypes. That section holds exactly one row — the product build — and a test compares it in both directions, so a second row fails the suite.

06 The Refinement form, field by field

This is what **Add link** opens on the full site.

REFINEMENT · ADD LINK

Address	The full <code>https://...</code> link. Required. Only <code>http</code> and <code>https</code> addresses are accepted — anything else is refused, on purpose, because every address here is rendered as a real link for everyone.
Title	What it is, in a few words. Required. Include the branch name so the row can be matched to a PR later: <i>“Study streak card — feat/study-streak-card”</i> .
Note	Optional, but use it. Say where to look and what question you want answered: <i>“Home, under Current Progress. Is the streak motivating or nagging?”</i>
Added by	Your name. Optional; remembered in your browser so you type it once. It is a label, not a login — nothing checks it.
Show on public site	A checkbox, off by default . The hint under it reads: <i>“Off: only the team sees this row. On: stakeholders with the public link see it too.”</i>

On the list, each row carries a chip — **Team only** or **Public** — so you can see at a glance what stakeholders can see. The nav badge on Refinement counts all rows on the full site and only the public ones on the public site.

There is no *Type* field here, unlike Other Links. Every Refinement row is the same kind of thing — work in review — and the toggle is what tells rows apart.

07 Things not to do

- **Don't push to `main`**. Every push to `main` rebuilds the public site. Work on a branch; get there through a PR.

- **Don't add a row to Prototypes** or change a row's `category` to `'prototype'`. Promotion is a flag default, never a tile move.
- **Don't put review HTML in `public/prototypes/`**. It is 404'd on the public build. Use `public/demos/`.
- **Don't change a flag's default on `main` yourself**. That is the promote step, and it is Jillienne's call.
- **Don't paste a stakeholder the full site's address**. They cannot open it, and the password is the team's. The review link is your branch build on the public site's domain.
- **Don't invent data**. A number, a URL, a lesson count or a fee that XCEL has not published is flagged as `INVENTED` in the code and named as such on screen. `CLAUDE.md` records why, several times over.
- **Don't draft the Quick Summary** on a dev-handoff component — ask Jillienne for it. The `dev-handoff-notes` skill says the same.

08 When something looks wrong

SYMPTOM	LIKELY CAUSE	TRY
Branch address 404s	Netlify hasn't finished the build, or branch deploys were switched off.	Wait three minutes and reload. If it still fails, check the branch name has slashes turned to dashes; then ask Jillienne to confirm branch deploys are on.
Branch shows old main, not your work	Your flag default is off on the branch, or you opened the page without <code>?demo=1</code> .	Add <code>?demo=1</code> . Then check <code>FEATURE_FLAGS</code> on your branch has your flag's default on.
The demo looks different on one laptop only	That browser holds a saved "Set as default" flag snapshot, which wins over the committed default per key.	In that browser's DevTools → Application → Local Storage, delete <code>cgp.featureFlags.customDefaults</code> .
Refinement says "endpoint unreachable"	You are on <code>npm run dev</code> locally, which serves	Add rows on the full site, not locally. (<code>netlify dev</code> works locally but needs setup; see <code>.claude/launch.json</code> .)

SYMPTOM	LIKELY CAUSE	TRY
	no <code>/api/</code> <code>*</code> .	
Row shows "not a linkable address"	The address wasn't <code>http(s)</code> , or was saved by an older version.	Edit the row and paste the full <code>https://</code> address.
Stakeholder can't see your row	<i>Show on public site</i> is off.	Edit the row → switch it on. Check the chip reads Public .
Tests fail on something you didn't touch	A decision is pinned — nav order, section contents, a colour that must keep failing.	Read the test's comment. Either your change re-decides it (update the test, say so) or it exposed a real regression.
<code>git</code> says <code>index.lock</code> exists	A previous git process died mid-write.	<code>rm .git/index.lock</code> — only if no other git command is running.

09 Glossary

Repo	The project's code, in git. <code>jillmoreno/xcel-ux-dashboard</code> on GitHub.
Clone	Download the repo to your machine, once.
Branch	Your own line of work inside the repo. Nothing on it is public until merged — but its <i>build</i> has a public address.
Push	Send your commits to GitHub. Netlify builds every pushed branch.
Branch deploy	The website Netlify builds for one branch: <code><branch>-ux-demo-xceldashboard.netlify.app</code> .
<code>?demo=1</code>	Demo mode: render the committed flag defaults, ignore the viewer's own saved toggles, don't persist flips.
Feature flag	A named switch in <code>FEATURE_FLAGS</code> . Off on <code>main</code> until promoted; on on your branch so your build shows it.
Prototypes	The section holding the live product build at <code>main</code> 's flag defaults. One row. Changed only by merging.
Refinement	The review inbox: links added on the page, team-only until flipped public. Its internal id is <code>demo</code> — you will see that in URLs (<code>?section=demo</code>) and code.
Other Links	Everything else that lives elsewhere: briefs, boards, Figma files. Also added on the page.
Promote	Jillienne's step on your PR: decide per flag whether its "on" default ships in Prototypes. Skill: <code>promote-to-prototype</code> .
Full site / public site	The password-protected everything site, and the passwordless stakeholder site. Same code; one build-time switch.

Part of the XCEL UX Dashboard. The reasoning behind every decision described here is in the repo's `CLAUDE.md`; the one-time site setup is in `README.md` under “Two Netlify sites, one repo”. Stakeholders get a shorter companion page, [How to read this dashboard](#).